

# CSIT314 — Software Development Methodologies

## Mini-Project 2 (System B) — Library Book Borrowing System

XXXXX

Instructor: XXXXX

| Student ID | First Name | Last Name | Sections      |
|------------|------------|-----------|---------------|
| XXXXXXX    | XXXXX      | XXXXX     | 4.1, 4.4      |
| XXXXXXX    | XXXXX      | XXXXX     | 4.2, 5        |
| XXXXXXX    | XXXXX      | XXXXX     | 4.5           |
| XXXXXXX    | XXXXX      | XXXXX     | 4.3, 4.4, 4.5 |

### 4.1 SYSTEM UNDERSTANDING

#### SYSTEM PURPOSE

The Library Books Borrowing System is a system developed to automate the core loan activities of a library. Members are able to check out books and return borrowed books, while librarians are able to update records accordingly. The system also tracks loan issuance dates, due dates, and outstanding overdues.

##### MAIN USERS

- **Member:** a registered patron who can check out and return books, with a loan history maintained against their account.
- **Librarian:** library staff who can manage book inventory, issue and record loan activity, and generate reports on loan status.

##### CORE FEATURES

- Checking out books and updating their availability status
- Issuing, tracking, and closing member loans
- Monitoring loan due dates and flagging overdues
- Generating book inventory and member loan reports

#### AMBIGUITY / DESIGN CONCERN

The Loan class has a status attribute and a `closeLoan()` method, however, there is no mechanism for overdue detection or escalation, which would lead to overdue loans remaining unresolved with no notification or alert triggered, the design records that something is overdue but takes no action on it.

The Report class has no association with the Loan class, and `reportType` is left as an open-ended untyped attribute resulting in no traceable path between report outputs and loan data, meaning a librarian generating a loan status report cannot be assured the report output is grounded in actual loan records as the design provides no mechanism to verify it.

### 4.2 REQUIREMENTS—DESIGN—TEST TRACEABILITY

#### WHAT IS A TRACEABILITY TABLE?

A traceability table is a tool used in software quality assurance that creates a clear link between the different components within a system and its requirements. It links the system requirements, design elements and testing activities to make sure that every requirement is accounted for in the design and has a corresponding test. This allows for identifying gaps within the system design that can be later targeted and improved upon.

## TRACEABILITY TABLE

| Requirement                                                                                     | UML Element                                                      | Test Type      | Test Objective                                                                                                                   |
|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>R1:</b> Member is able to borrow an available book                                           | Member.borrowBook(),<br>Book.markBorrowed(),<br>Loan             | Black-box      | This will confirm the loan is completed successfully and the book's status will be updated to Borrowed                           |
| <b>R2:</b> Member is able to return a borrowed book                                             | Member.returnBook(),<br>Book.markReturned(),<br>Loan.closeLoan() | Black-box      | This will confirm that the loan closes and the book's status is updated to Available                                             |
| <b>R3:</b> When the loan is created, the system records the borrow and due date                 | Loan.borrowDate,<br>Loan.dueDate                                 | Black-box      | This confirms that both of the dates are stored correctly when issued                                                            |
| <b>R4:</b> The system is expected to detect and escalate overdue loans                          | Loan.status,<br>Loan.dueDate <sup>1</sup>                        | Boundary value | This exposes the absence of overdue escalation beyond just status flagging                                                       |
| <b>R5:</b> The book's availability status transitions correctly across the different operations | Book.markBorrowed(),<br>Book.markReturned()                      | White-box      | This verifies the internal state logic follows the correct branch on every operation                                             |
| <b>R6:</b> When a loan is closed, the status is updated without any side effects                | Loan.closeLoan(),<br>Loan.status                                 | White-box      | This verifies the method that sets the status to closed, which stops any further state changes                                   |
| <b>R7:</b> A librarian is able to generate a report that reflects the current loan data         | generateReport() <sup>2</sup>                                    | Failure        | This exposes that the report output cannot be verified against loan data as there's a missing class association                  |
| <b>R8:</b> The system correctly handles a borrow request when a book is unavailable             | Book.status,<br>Member.borrowBook()                              | Boundary value | This confirms that the system can correctly enforce the availability boundary, and rejecting the request when status is borrowed |
| <b>R9:</b> A librarian is able to update book inventory records                                 | Book(bookID, title, author, status) <sup>3</sup>                 | Black-box      | This confirms that the book details are able to be updated and reflected within the whole system                                 |
| <b>R10:</b> The system maintains loan history per member                                        | Member,<br>Loan (1 member → Many loans) <sup>4</sup>             | Black-box      | This confirms that all previous loans are still retrievable for a member's account                                               |

<sup>a</sup> No escalation method defined.<sup>b</sup> No association to Loan defined.<sup>c</sup> No explicit method is defined in baseline class.<sup>d</sup> Relationship is implied but no method is defined.

Figure 1: Requirements Traceability Matrix

The traceability table above maps functional requirements that were gathered from the existing Library Book Borrowing system, to its UML counterparts, test type and objectives. This fully covers the system's core features that were identified in Section 4.1. This also exposes gaps in the baseline design, for instance the absence of explicit methods for overdue escalations and loan history retrieval, which are carried forward for analysis in later sections.

### 4.3 TEST CASE DESIGN

---

To test whether the system performs as expected and fulfills user requirements, we formulate test cases to test different aspects of the system. We specifically create 4 black-box, 2 white-box, 1 boundary, and 1 failure test to ensure all components are covered. These are inline with the traceability table in 4.2 and serve to further elaborate on, and create more concrete testing conditions. To make up for the incomplete system requirements, we also detail various system and testing assumptions to fill in the gaps.

Black-box test cases are presented in the following order: a table detailing input, procedure and expected outcome of the test-case; an equivalency partitioning table; and a sample inputs and outputs table. For boundary value analysis (BVA), as there are 2 requirements for this test type in the traceability, we decided on R4 as it could be more easily classified into boundaries, and allowed us to address a potential loophole in the return and borrowing functions. As for negative/failure test-cases, we identified several areas where invalid inputs could lead to errors, and compiled them together in a table. Finally, for the white-box test cases, we followed R5 and R6 to implement the `closeLoan()`, `markBorrowed()`, and `markReturned()` functions in Python. We presented all the possible paths that can be taken using a CFG (Control Flow Graph) alongside the test-cases table.

#### SYSTEM & TESTING ASSUMPTIONS

|                      |                                                                                                                                                                                                                                                                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Assumption 1</b>  | The possible types of reports that can be generated are: loan history of a member (LHM), loan history of a book (LHB), loan history of all members and books (LHBM), and member profile (MP).                                                                                                                                            |
| <b>Assumption 2</b>  | If any of a member's loan's dueDate has been passed ( <code>currentDate &gt; dueDate</code> ), the member cannot borrow anymore books until the book is returned, and the according fine is paid (MP report created by <code>generateReport()</code> ).                                                                                  |
| <b>Assumption 3</b>  | If there are no outstanding loans ( <code>Loan(status) != overdue</code> ), a member can loan out a maximum of 5 books at any given time. Each book can be loaned for days at a time before it is marked as "overdue".                                                                                                                   |
| <b>Assumption 4</b>  | A valid ID of any class follows the following format: ID type indicator (b, m, l, r) + 7 digits (0-9).                                                                                                                                                                                                                                   |
| <b>Assumption 5</b>  | IDs are automatically input to whichever needed function to avoid manual errors (e.g. ID does not exist).                                                                                                                                                                                                                                |
| <b>Assumption 6</b>  | The possible status of a Loan are: "borrowed", "returned", or "overdue"; a Loan is created when <code>borrowBook()</code> is called, and subsequently logged for report generation and monitoring.                                                                                                                                       |
| <b>Assumption 7</b>  | The possible status of a book are: "available", "unavailable", or "maintenance" (for debug cases, replacing a lost book, etc.).                                                                                                                                                                                                          |
| <b>Assumption 8</b>  | The system automatically changes loan status from "borrowed" to "overdue" when dueDate is passed.                                                                                                                                                                                                                                        |
| <b>Assumption 9</b>  | Fines automatically accrue overtime; payment is handled by a third-party system and is not within the scope of these test cases.                                                                                                                                                                                                         |
| <b>Assumption 10</b> | Two additional methods are added to the Loan class: <code>createLoan()</code> which is called by <code>borrowBook()</code> and initializes the class variables, and <code>prevLoan()</code> which fetches the previous Loan made by the same member for the same book as a new Loan instance is created ( <code>default = None</code> ). |
| <b>Assumption 11</b> | The attribute "returnDate" is added to the Loan class to log the date at which a loan is closed ( <code>default = None</code> until Loan is closed).                                                                                                                                                                                     |

## BLACK-BOX TEST CASES

### TEST CASE 1

|                         |                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requirement</b>      | <b>R1:</b> Member is able to borrow an available book                                                                                                                                                                                                                                                                                                                                   |
| <b>Purpose</b>          | To ensure system logic flows correctly for borrowBook() (changing book states with Loan creation and Loan closure) to enable members to successfully borrow and return books, with all system and member actions accurately logged and updated.                                                                                                                                         |
| <b>Input</b>            | BookID & status of book to borrow<br>MemberID and LHM of borrowing member                                                                                                                                                                                                                                                                                                               |
| <b>Procedure</b>        | <ol style="list-style-type: none"> <li>1. Check the status of the book with BookID; if status != "available", return "Book currently unavailable/under maintenance" error.</li> <li>2. Generate LHM of member with MemberID; if any loan is "overdue", return "Loan overdue: unable to borrow new books" error.</li> <li>3. If conditions 1 &amp; 2 pass, call borrowBook().</li> </ol> |
| <b>Expected Outcome</b> | Book status changed to "borrowed" & markBorrowed() called, new Loan instance created for member & logged to system; any errors handled as per the procedure.                                                                                                                                                                                                                            |

Figure 2: Black-box Test Case 1 – R1

| Input                 | Valid Class                   | Invalid Class                                |
|-----------------------|-------------------------------|----------------------------------------------|
| Book(BookID)          | b + 7 digits (0–9)            | Different ID type indicator or non-digits    |
| Book(status)          | "available"                   | "unavailable" or "maintenance"               |
| Member(MemberID)      | m + 7 digits (0–9)            | Different ID type indicator or non-digits    |
| Report(LHM, MemberID) | No loan with status "overdue" | At least a single loan with "overdue" status |

Figure 3: Equivalence Partitioning Classes – R1

| Test Case | BookID   | status        | MemberID | LHM           | Validity                                |
|-----------|----------|---------------|----------|---------------|-----------------------------------------|
| <b>C1</b> | b1234567 | "available"   | m1234567 | No "overdue"s | Valid                                   |
| <b>C2</b> | b12a4567 | "available"   | m1234567 | No "overdue"s | Invalid; only digits accepted           |
| <b>C3</b> | b1234567 | "unavailable" | m1234567 | No "overdue"s | Invalid; book unavailable               |
| <b>C4</b> | b1234567 | "available"   | m123456  | No "overdue"s | Invalid; MemberID != 6 digits           |
| <b>C5</b> | b1234567 | "available"   | m1234567 | One "overdue" | Invalid; return overdue book & pay fine |

Figure 4: Test Cases – R1

## TEST CASE 2

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requirement</b>      | <b>R2:</b> Member is able to return a borrowed book                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Purpose</b>          | To ensure system logic flows correctly for <code>returnBook()</code> by checking that the function correctly calls <code>markReturned()</code> , changes status to "available", and handles <code>closeLoan()</code> .                                                                                                                                                                                                                                                                                                                                                         |
| <b>Input</b>            | BookID of book to return<br>LoanID & status of loan<br>MemberID of returning member                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Procedure</b>        | <ol style="list-style-type: none"> <li>1. Generate LHM of the returning member with MemberID, check Loan with LoanID.</li> <li>2. If Loan status is "overdue", prompt the member to pay a fine amount through a third-party handler (any errors also handled by the payment system). Change Loan status to "borrowed" when fine is paid.</li> <li>3. If all outstanding fines are paid and the book is physically returned, call <code>closeLoan()</code> to change the status of Loan to "returned" and call <code>markReturned()</code> for the book with BookID.</li> </ol> |
| <b>Expected Outcome</b> | Book status changed to "available", <code>markReturned()</code> & <code>closeLoan()</code> called, fines successfully paid, Loan status changed to "returned" & logged to system; any errors handled.                                                                                                                                                                                                                                                                                                                                                                          |

Figure 5: Black-box Test Case 2 – R2

| Input            | Valid Class        | Invalid Class                             |
|------------------|--------------------|-------------------------------------------|
| Book(BookID)     | b + 7 digits (0–9) | Different ID type indicator or non-digits |
| Loan(LoanID)     | l + 7 digits (0–9) | Different ID type indicator or non-digits |
| Loan(status)     | "borrowed"         | "returned" or "overdue"                   |
| Member(MemberID) | m + 7 digits (0–9) | Different ID type indicator or non-digits |

Figure 6: Equivalence Partitioning Classes – R2

| Test Case | BookID   | LoanID   | status     | MemberID | Validity                      |
|-----------|----------|----------|------------|----------|-------------------------------|
| <b>C1</b> | b1234567 | l1234567 | "borrowed" | m1234567 | Valid                         |
| <b>C2</b> | b12a4567 | l1234567 | "borrowed" | m1234567 | Invalid; only digits accepted |
| <b>C3</b> | b1234567 | m1234567 | "borrowed" | m1234567 | Invalid; wrong type indicator |
| <b>C4</b> | b1234567 | l1234567 | "overdue"  | m1234567 | Invalid; fine unpaid          |
| <b>C5</b> | b1234567 | l1234567 | "borrowed" | m123456  | Invalid; MemberID != 6 digits |

Figure 7: Test Cases – R2

### TEST CASE 3

|                         |                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requirement</b>      | <b>R3:</b> When the loan is created, the system records the borrow and due date.                                                                                                                                                                                                                                                                                           |
| <b>Purpose</b>          | To ensure that borrowBook() correctly calls the createLoan() method with correctly defined borrow and due dates.                                                                                                                                                                                                                                                           |
| <b>Input</b>            | BookID, status, currentDate                                                                                                                                                                                                                                                                                                                                                |
| <b>Procedure</b>        | <ol style="list-style-type: none"> <li>1. When borrowBook() is called, call createLoan() with the following initial variables: new randomized LoanID, status = "borrowed", borrowDate = currentDate, dueDate = currentDate + 15 days.</li> <li>2. Log creation event in system, mark status of book with BookID to "unavailable". Link loan to member and book.</li> </ol> |
| <b>Expected Outcome</b> | Initialized loan instance with correctly initialized dates, status of book being borrowed set to "unavailable", loan appears on generated reports.                                                                                                                                                                                                                         |

Figure 8: Black-box Test Case 3 – R3

| Input        | Valid Class                             | Invalid Class                                              |
|--------------|-----------------------------------------|------------------------------------------------------------|
| Book(BookID) | b + 7 digits (0–9)                      | Different ID type indicator or non-digits                  |
| Book(status) | "available"                             | "unavailable"                                              |
| currentDate  | Datetime format<br>2022-05-23T20:31:08Z | (e.g. Date past currentDate, format different to datetime) |

Figure 9: Equivalence Partitioning Classes – R3

| Test Case | BookID   | status        | currentDate          | Validity                                |
|-----------|----------|---------------|----------------------|-----------------------------------------|
| <b>C1</b> | b1234567 | "available"   | 1995-11-12T20:32:08Z | Valid                                   |
| <b>C2</b> | b12a4567 | "available"   | 2010-08-23T14:09:04Z | Invalid; only digits accepted           |
| <b>C3</b> | b1234567 | "unavailable" | 2026-06-24T18:27:03Z | Invalid; cannot borrow unavailable book |
| <b>C4</b> | b1234567 | "available"   | 2027-06-24T18:27:03Z | Invalid; date is past currentDate       |

Figure 10: Test Cases – R3

#### TEST CASE 4

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requirement</b>      | <b>R9:</b> A librarian is able to update book inventory records.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Purpose</b>          | For librarians to be able to update details and place books under maintenance as needed for book records to be kept accurate.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Input</b>            | BookID<br>Per update, possible inputs are: title, author, status                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Procedure</b>        | <ol style="list-style-type: none"> <li>1. Librarian selects a book to update using BookID. The option to modify (title/author/status) is selected and the new value entered.</li> <li>2. If the title or author is changed, no user logs or reports change, as they are based on BookID. The new title/author is displayed and the changes logged in the system.</li> <li>3. If the status is changed to "maintenance" (no alternative option), the book is removed from member view. If it was previously "unavailable", the Loan associated with it is closed. All changes are logged in the system.</li> </ol> |
| <b>Expected Outcome</b> | Correct entered input type modified and the changes displayed, status of book changed and associated Loan closed, all changes logged.                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

Figure 11: Black-box Test Case 4 – R9

| Input        | Valid Class                     | Invalid Class                               |
|--------------|---------------------------------|---------------------------------------------|
| Book(BookID) | b + 7 digits (0–9)              | Different ID type indicator or non-digits   |
| Book(title)  | A–Z, a–z, special chars, digits | Any char not following UTF-8 format         |
| Book(author) | A–Z, a–z, spaces, apostrophe    | Any other char (e.g. digits, special chars) |
| Book(status) | "available" or "unavailable"    | "maintenance"                               |

Figure 12: Equivalence Partitioning Classes – R9

| Test Case | BookID   | title                   | author               | status        | Validity                        |
|-----------|----------|-------------------------|----------------------|---------------|---------------------------------|
| <b>C1</b> | b1234567 | A history of science <3 | Ibn Khattab          | "available"   | Valid                           |
| <b>C2</b> | b12a4567 | A history of history!!! | Tolstoy              | "unavailable" | Invalid; only digits accepted   |
| <b>C3</b> | b1234567 | A history of 0xc0       | Flower Pot           | "unavailable" | Invalid; non UTF-8 char used    |
| <b>C4</b> | b1234567 | A history of cooking && | Hee/ee               | "available"   | Invalid; / char used            |
| <b>C5</b> | b1234567 | A history of carpets3   | Sduhgfuisd sd-jfgisd | "maintenance" | Invalid; already in maintenance |

Figure 13: Test Cases – R9

## BOUNDARY VALUE TEST

| Equivalency Partitioning Table for R4                                                                                                                                                                                                                                                                                                                                        |                                                          |                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------|
| <b>Test Purpose:</b> This test-case specifically deals with the detection aspect of R4: determining whether a loan is overdue, by checking whether the number of days since the borrow date has exceeded 15. It also checks whether the member is abusing the return function to re-borrow a book immediately after it was returned by introducing <code>prevLoan()</code> . |                                                          |                                                         |
| Invalid                                                                                                                                                                                                                                                                                                                                                                      | Valid                                                    | Invalid                                                 |
| <code>Loan.prevLoan().returnDate = Loan.borrowDate</code>                                                                                                                                                                                                                                                                                                                    | <code>abs(currentDate - Loan.borrowDate) &lt;= 15</code> | <code>abs(currentDate - Loan.borrowDate) &gt; 15</code> |

Figure 14: Equivalence Partitioning Classes – R4

| Test Case | Loan.prevLoan().return | Loan.borrowDate | currentDate | Validity                          |
|-----------|------------------------|-----------------|-------------|-----------------------------------|
| C1        | None                   | 2026-09-05      | 2026-09-10  | Valid                             |
| C2        | 2026-06-24             | 2026-09-05      | 2026-09-21  | Invalid; overdue                  |
| C3        | 2026-09-05             | 2026-09-05      | 2026-09-05  | Invalid; same day return & borrow |

*NOTE – YYYY-MM-DD format is used in this table as opposed to datetime solely for better presentability and comprehension.*

Figure 15: Test Cases – R4

## FAILURE / NEGATIVE TEST

### Failure Test Cases Table

| Test Case | Input                                                                           | Expected Output                                                             |
|-----------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| C1        | Librarian attempts to generate a report without specifying any attributes.      | Missing Loan/Member associations; please specify them.                      |
| C2        | Member attempts to re-borrow a book returned on the same day.                   | Cannot re-borrow book; please try again another day.                        |
| C3        | Member attempts to borrow a book with status "unavailable" or "maintenance".    | Cannot borrow book; please try again when book status = "available".        |
| C4        | Member attempts to borrow a book while one of their Loans has "overdue" status. | Cannot borrow book; please return [BookID] book and pay the according fine. |
| C5        | Missing member/loan/book ID depending on the function called.                   | Missing [missing ID]; please enter the [missing ID] to try again.           |
| C6        | Member/loan/book ID entered does not exist in the database.                     | [ID] does not exist; please enter another [ID type].                        |

Figure 16: Test Cases – R7 (Failure)



## WHITE-BOX TEST CASES

### TEST CASE 1

**Test Purpose:** To check that a book's availability status transitions correctly from "available" to "unavailable" when borrowed, and from "unavailable" to "available" when returned.

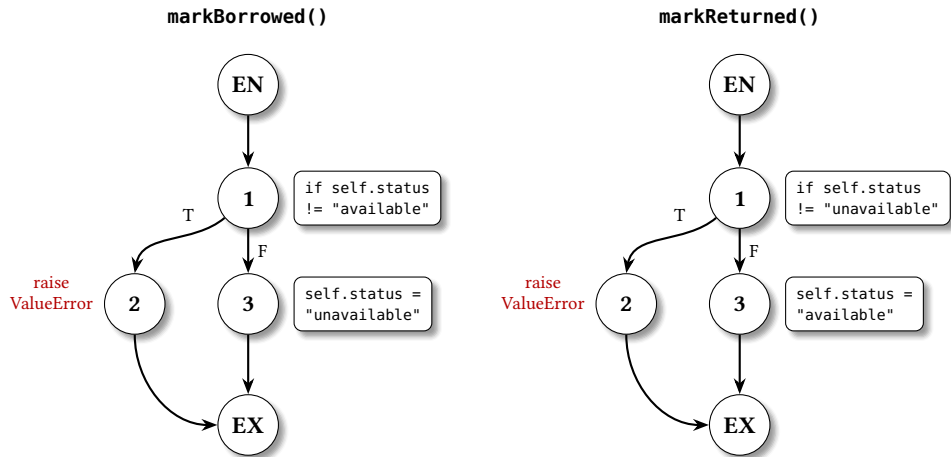
| Test Case | Method         | Path Covered             | Input (Book.status)            | Expected Output                                         |
|-----------|----------------|--------------------------|--------------------------------|---------------------------------------------------------|
| C1        | markBorrowed() | EN -> 1(F) -> 3<br>-> EX | "available"                    | Valid; status set to "unavailable", no exception raised |
| C2        | markBorrowed() | EN -> 1(T) -> 2<br>-> EX | "unavailable" or "maintenance" | Invalid; status unchanged, ValueError raised            |
| C3        | markReturned() | EN -> 1(F) -> 3<br>-> EX | "available"                    | Valid; status set to "available", no exception raised   |
| C4        | markReturned() | EN -> 1(T) -> 2<br>-> EX | "unavailable" or "maintenance" | Invalid; status unchanged, ValueError raised            |

Figure 17: Test Cases — R5 (White-box)

#### Code Editor

```
1  class Book:
2      def __init__(self, book_id: str, title: str, author: str):
3          self.bookID = book_id
4          self.title = title
5          self.author = author
6          self.status = "available" # "available" | "unavailable" | "maintenance"
7
8      def markBorrowed(self):
9          if self.status != "available":
10             raise ValueError(f"Cannot borrow book; status is '{self.status}'")
11             self.status = "unavailable"
12
13      def markReturned(self):
14          if self.status != "unavailable":
15             raise ValueError(f"Cannot return book; status is '{self.status}'")
16             self.status = "available"
```

Figure 18: Code — R5 (White-box)



## TEST CASE 2

**Test Purpose:** To check that a Loan's status transitions correctly depending on its initial state, and to check for any fines that may need to be paid before a Loan is closed.

| Test Case | Path Covered                                  | Input (Loan.status) | Expected Output                                                                                                          |
|-----------|-----------------------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------|
| C1        | EN -> 1(F) -><br>3(F) -> 5 -> 6<br>-> 7 -> EX | "borrowed"          | Valid; change Book.status to "available" & Loan.status to "returned", set returnDate to currentDate, no exception raised |
| C2        | EN -> 1(T) -><br>2 -> EX                      | "returned"          | Invalid; Loan is already closed, status and returnDate remain unchanged, raise ValueError                                |
| C3        | EN -> 1(F) -><br>3(T) -> 4 -><br>EX           | "overdue"           | Invalid; outstanding fine, returnDate unchanged, markReturned() not called                                               |

Figure 19: Test Cases – R6 (White-box)

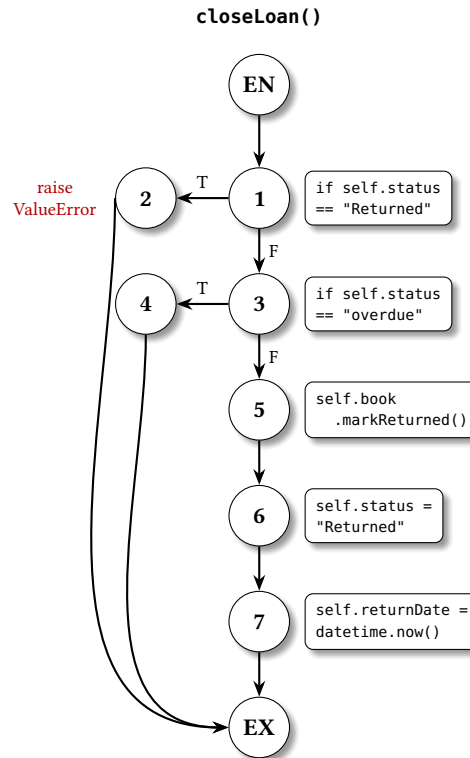
### Code Editor

```

3  class Loan:
4      def __init__(self, loan_id: str, book: Book, member_id: str, borrow_date):
5          self.loanID = loan_id
6          self.book = book
7          self.memberID = member_id
8          self.status = "borrowed" # "borrowed" | "returned" | "overdue"
9          self.borrowDate = borrow_date
10         self.dueDate = borrow_date + timedelta(days=15)
11         self.returnDate = None # Set on close
12
13     def closeLoan(self):
14         if self.status == "returned":
15             raise ValueError("Loan is already closed.")
16         if self.status == "overdue":
17             raise ValueError("Cannot close loan; outstanding fine must be paid first.")
18
19         self.book.markReturned()
20         self.status = "returned"
21         self.returnDate = datetime.now()

```

Figure 20: Code – R6 (White-box)



#### 4.4 VERIFICATION VS. VALIDATION

##### VERIFICATION

Verification confirms that the implementation of the system is as designed and specified, they include:

|                              |                                                                                                                                                                                                                           |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>UML Structural Review</b> | Inspecting sequence diagrams and class diagrams in order to confirm that associations (e.g. Member → Loan) are realised in the code.                                                                                      |
| <b>Unit Testing</b>          | Testing class methods individually in order to verify they do their specific behavior, for example that the <code>Book.status</code> transitions occur in the right sequence (Available → Borrowed → Available).          |
| <b>Integration Testing</b>   | Testing to confirm that functions correctly change the state they are connected to, for example the <code>borrowBook()</code> function updating Book, Member, and Loan classes.                                           |
| <b>Traceability Review</b>   | Verifying each requirement in a traceability table against a corresponding design and flagging gaps, an example of a gap that exists here is an undefined method for loan history retrieval (REQ: retrieve loan history). |

## VALIDATION

Validation confirms that the system satisfies all user needs and expectations, they include:

|                                      |                                                                                                                                                                                                                       |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User Acceptance Testing (UAT)</b> | Having librarians and members go through the workflows, borrowing books, returning books, viewing loan history etc. in order to confirm that the system behaves as expected in practice.                              |
| <b>Functional Scenario Testing</b>   | Simulating real conditions, for example a member attempting to borrow an unavailable book, in order to verify the system responds properly and matches operational requirements (checks status, rejection message).   |
| <b>Report Validation</b>             | Having librarians review the generated reports from the system; this would expose whether the report output reflects the actual loan data, which is particularly important given the missing Report→Loan association. |
| <b>Overdue Workflow Walkthrough</b>  | Walking through how the system handles overdue books with library staff to see if it aligns with their process expectations.                                                                                          |

## REALISTIC ESCAPED DEFECT

### DEFECT: REPORTS SILENTLY OMIT ACTIVE OR HISTORICAL LOAN DATA

**Mechanism:** There is no association between the Report class and the Loan class in the class diagram, therefore any report generation logic would have to access loan data indirectly (e.g., through Librarian), or rely on a workaround not captured in the design.

The defect is found when doing User Acceptance Testing (UAT), when a librarian generates a report against a dataset and observes that things such as loan counts, overdue figures, or borrowing histories are incorrect.

This is a validation escape, where the system is built as per a flawed design, which leads to requirements not being satisfied.

## 4.5 CI / CD & QUALITY REFLECTION

### WHAT IS CI / CD & QUALITY ASSURANCE?

#### CONTINUOUS INTEGRATION & CONTINUOUS DELIVERY (CI / CD)

Continuous Integration/Continuous Delivery (CI/CD) is a set of software engineering practices that automate the building, testing, and delivery of code so that changes can be integrated and released frequently in small increments as opposed to large batch updates [1].

It requires developers to merge their code changes into a shared repository frequently, with each merge auto triggering a build and automated tests so that integration problems are caught within minutes rather than discovered weeks later during a large merge [2].

Continuous Delivery (CD) builds on CI by auto packaging code that has passed its tests and keeping it in a constantly deployable, “evergreen” state, so that a working release can be sent at any time without a manual integration phase [1].

Continuous Deployment goes one step further and automatically pushes every change that passes the pipeline straight into production, with Continuous Delivery differing mainly in that it still requires a human decision before the final release [3].

A central feature of any CI/CD pipeline is its use of automation to ensure that code is at peak quality at every stage [3].

Since defects, security issues, and design flaws are identified as early as possible in the development cycle, they are cheapest and easiest to fix, rather than being discovered after deployment [4].

#### SOFTWARE QUALITY ASSURANCE (QA)

Quality Assurance is the process of ensuring software is built and maintained in a way that consistently meets defined quality standards [5]. A useful distinction is that QA is concerned with preventing defects by improving the process used to build the software, whereas quality control is about finding defects in a finished product [5].

#### QUALITY GATES

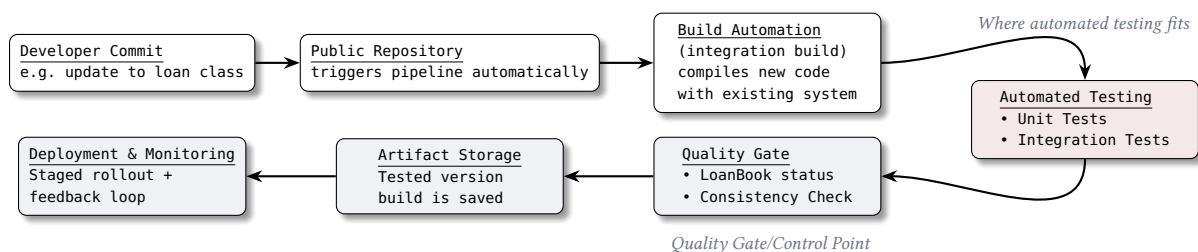
A quality gate is a checkpoint inside the pipeline where the build is evaluated against a standard before it is allowed to proceed to the next stage [6]. If the build does not meet the standard, the pipeline stops there rather than letting the build continue toward deployment.

### CI PIPELINE ASSUMPTIONS

The following assumptions were made to reason how the system would move through a CI/CD pipeline:

- Assumption 1** An integration build automation process is used for the system.
- Assumption 2** Unit and integration tests run during the automated testing stage.
- Assumption 3** A rolling/staged deployment approach is used, where a build is released gradually rather than to all users at once.
- Assumption 4** The pipeline itself does not experience a failed build caused by the diagram.

### CONCEPTUAL CI PIPELINE



*A failed test or unmet quality gate pauses the pipeline until the issue is successfully resolved.*

Figure 21: CI / CD Pipeline Flow Chart

### PIPELINE WALK-THROUGH

The pipeline begins when a developer commits a code change. For example, a fix to `Loan.closeLoan()` or `Book.markBorrowed()` to the shared central repository. This commit auto triggers the pipeline, which erases the necessity of a human to manually kick off testing.

In the **integration build** stage, the new code is compiled together with the existing system to confirm there are no integration errors. For example, a change to the `Loan` class does not break any of its links to `Member` and `Book`.

| Stage                 | Purpose                                                                                                                                                              |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Developer Commit      | A developer commits a code change (e.g. to <code>Loan.closeLoan()</code> or <code>Book.markBorrowed()</code> ) to the shared repository, auto triggers the pipeline. |
| Public Repository     | The commit is registered in the central repository, so the change is made visible to the whole team.                                                                 |
| Integration Build     | The new and improved code is compiled together with the existing system to confirm there are no compilation or integration errors.                                   |
| Automated Testing     | Unit and integration tests run automatically to confirm the change behaves correctly on its own and in order with the rest of the system.                            |
| Quality Gate          | The build is checked against predefined quality standards before it is allowed to proceed; if it fails, the pipeline halts.                                          |
| Artifact Storage      | The verified build is stored so that the exact tested version is the one that gets deployed.                                                                         |
| Deployment            | The stored build is released to the live environment.                                                                                                                |
| Monitoring & Feedback | The deployed system is observed for issues, and feedback is fed back to developers to guide future changes.                                                          |

Figure 22: CI/CD Pipeline Stages

## WHERE AUTOMATED TESTING FITS

Automated testing runs immediately after the build, and is where most of the defects are caught. Unit tests target individual methods in isolation. Such as: `Book.markBorrowed()`, `Member.borrowBook()`, and `Loan.closeLoan()` checking that each method correctly updates status on its own. Integration tests assure that the classes work together flawlessly. For instance, borrowing a book creates a `Loan` correctly linked to the right `Member` and `Book`, and that returning a book updates both the loan status and the book status together, rather than just one of the two. If any of the test fail, the pipeline stops before applying any changes and progressing further.

## QUALITY GATE

After testing, a quality gate checks that all the tests are passed and the build meets the standard. For this system, a quality gate would specifically check that no loan can be closed while its status indicates an outstanding overdue. This is the exact kind of necessary check that should block a release at this control point, rather than it be left until after it has been released. Once the gate is cleared successfully, the build is then stored in artifact storage, and the test version is ready for release. Release will follow a staged deployment strategy, with necessary monitoring to catch any issues that surface to daily usage of the system.

## DESIGN OR PROCESS WEAKNESS

The baseline design has no mechanism for overdue detection. The `Loan` class has a `status` attribute and a `closeLoan()` method, but nothing in the design updates status when `dueDate` passes and is overdue, and there is no method that notifies a member or restricts their account once a loan becomes overdue. Automated testing can only verify what the design says should happen, it cannot test for a feature that was never specified. This means a real flaw (where overdue loans go unflagged) could pass every automated test in the pipeline and reach the deployment stage completely undetected.

## POSSIBLE IMPROVEMENTS

A way to possibly improve this system would be implementing a check process that takes care of overdue dates which is either a scheduled job or a method such as `Loan.checkOverdue()` which compares `dueDate` against the

current date and updates the status accordingly. This would be just a process addition, not a redesign of the classes that already exist.

Once this method is brought into existence, it can be implemented directly into the CI which was described above: a unit test would confirm that the status changes correctly at and past the due date, and the existing quality gate could be extended to confirm that overdue loans are flagged before any report is then generated. This would close the gap between what the design currently records and what action the system takes, and it gives the pipeline something solid to test where previously there was nothing.

## 5. CONCLUSION

---

Overall, this report has provided an analysis of the Library Book Borrowing system across 4 areas: system understanding, traceability, test design and quality assurance practices.

We have established that the system successfully covers the core functionalities of borrowing and return workflows, with a clear structured class model (the different functions associated with each) as well as clear role separations between Member and Librarian.

However, there are two structural gaps that were consistently identified throughout the analysis:

- The absence of an overdue escalation mechanism in the Loan class
- The missing association between Report and Loan

These mistakes show up in the traceability matrix as requirements that cannot be tested. They also appear in failure and boundary test cases as weaknesses that can be exploited. The automated testing in the CI/CD pipeline does not catch these problems because it cannot check for behaviour that was never specified.

The verification and validation segment reinforces that the system can be built exactly as designed yet still fail user needs and expectations. In addition, the CI/CD pipeline section shows us that tools alone cannot fix problems at a design level – quality gates can only ensure that what has been defined is followed.

The improvements that we suggest implementing are as follows:

- Add a `Loan.checkOverdue()` method
- Establish a formal `Report → Loan` association

Although minimal, these changes would significantly close both gaps previously mentioned, making the system testable and functionally complete.

## REFERENCES

---

- [1] IBM. “What is continuous integration?” Accessed: 21 June 2026. 2026. URL: <https://www.ibm.com/think/topics/continuous-integration>.
- [2] Cisco. “What is CI/CD? — Continuous Integration and Continuous Delivery”. Accessed: 21 June 2026. 2026. URL: <https://www.cisco.com/site/us/en/learn/topics/computing/what-is-ci-cd.html>.
- [3] IBM. “Continuous integration/continuous delivery (CI/CD)”. Accessed: 21 June 2026. 2026. URL: <https://www.ibm.com/think/topics/ci-cd>.
- [4] Datadog. “What is CI/CD (Continuous Integration/Continuous Deployment)?” Accessed: 21 June 2026. 2026. URL: <https://www.datadoghq.com/knowledge-center/continuous-integration-deployment/>.
- [5] IBM. “Finding the right balance in AI-assisted QA in software testing”. Accessed: 22 June 2026. 2026. URL: <https://www.ibm.com/think/insights/quality-assurance-in-software-testing-with-ai>.
- [6] SonarSource. “Integrating quality gates into your CI/CD pipeline”. Accessed: 22 June 2026. 2024. URL: <https://www.sonarsource.com/resources/library/integrating-quality-gates-ci-cd-pipeline/>.